

REVERSE ENGINEERING BOOTCAMP

23RD-26TH SEPT

02:00 PM BST



UNIVERSITY
OF LONDON
GLOBAL
COMMUNITY

CYBERSECURITY SOCIETY

What to expect?

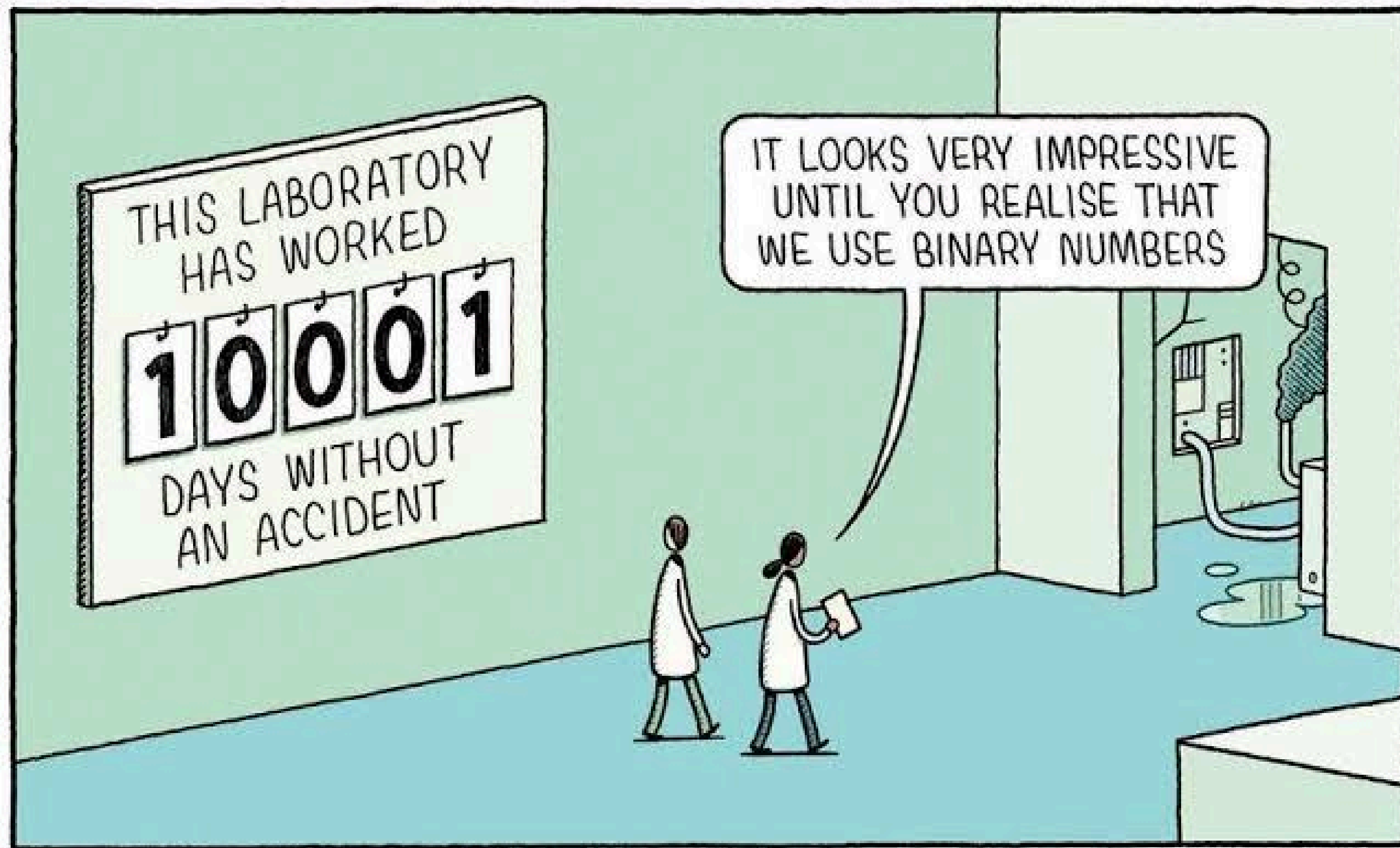
- **Day 1:** x86-64 assembly
- **Day 2:** Introduction to Ghidra and IDA
- **Day 3:** Live debugging with GDB
- **Day 4:** Putting everything together
- Feedback

Agenda: Day 1

- Brief history of Assembly
- Memory Hierarchy
- Registers
- The Stack
- x86-64 Assembly Language: Part 1
- x86-64 Assembly Language: Part 2
- Discussion and Resources

Brief history of Assembly

- In the olden days, computers were essentially number crunching machines.
- Computers can only understand binary language, but humans find it extremely tedious.
- Kathleen Booth co-wrote the Assembly Language to make programming easier.
- Today, Assembly Language is an integral part of reverse engineering.



TOM GAULD for NEW SCIENTIST

Memory Hierarchy



Bank (Slowest)



Safe (Slow)



Cash register (Fast)



Wallet
(Fastest)

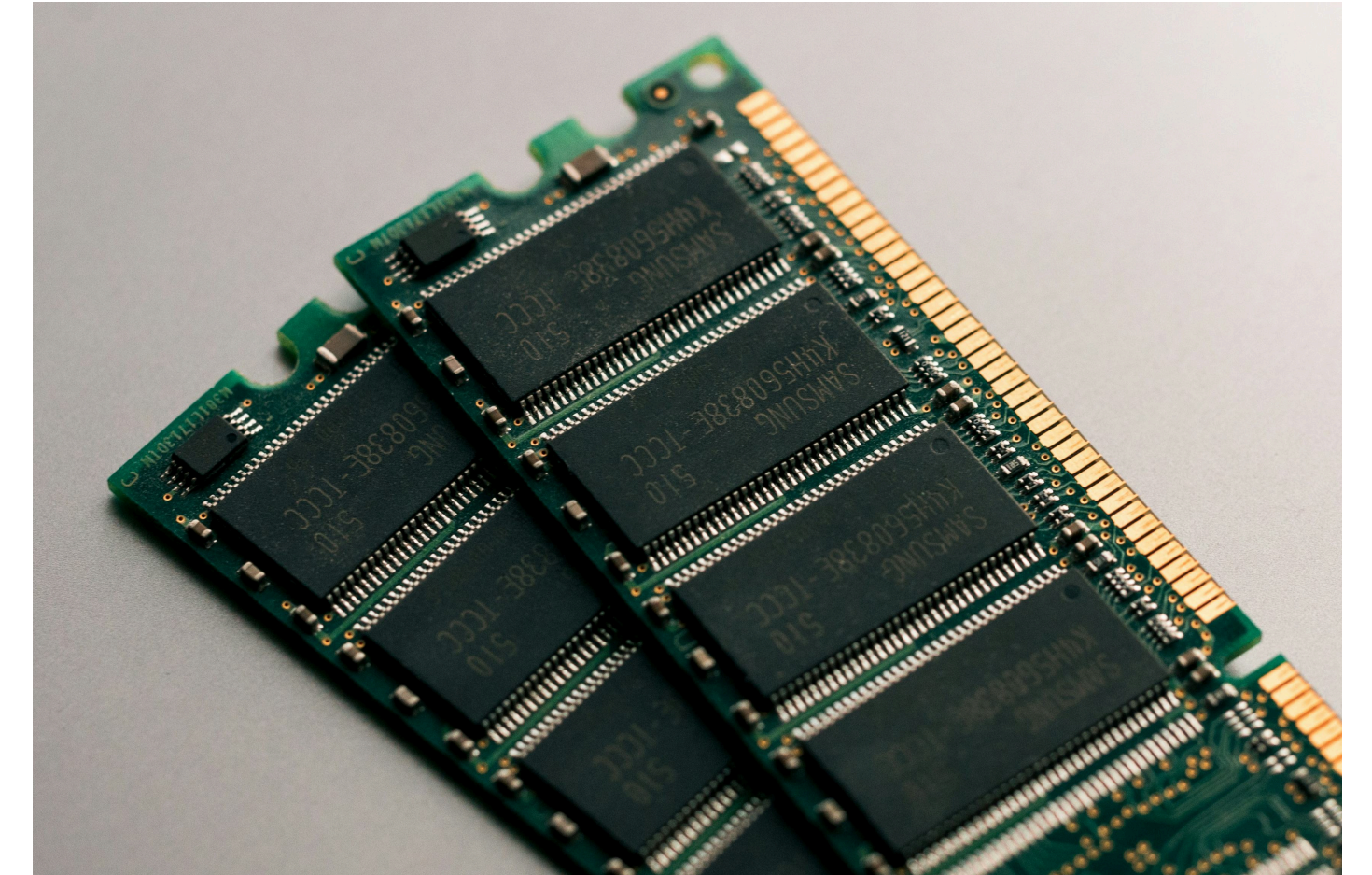
Memory Hierarchy (Computers)



External Disks (Slowest)



HDD/SSD (Slow)



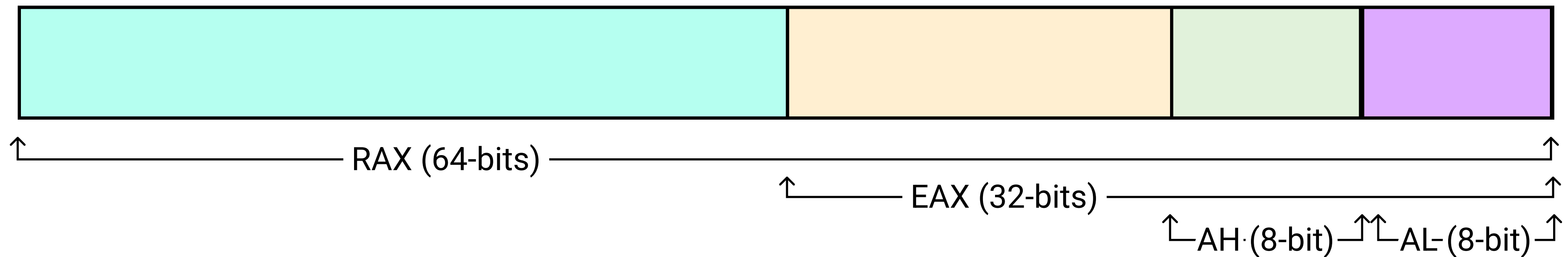
RAM (Fast)



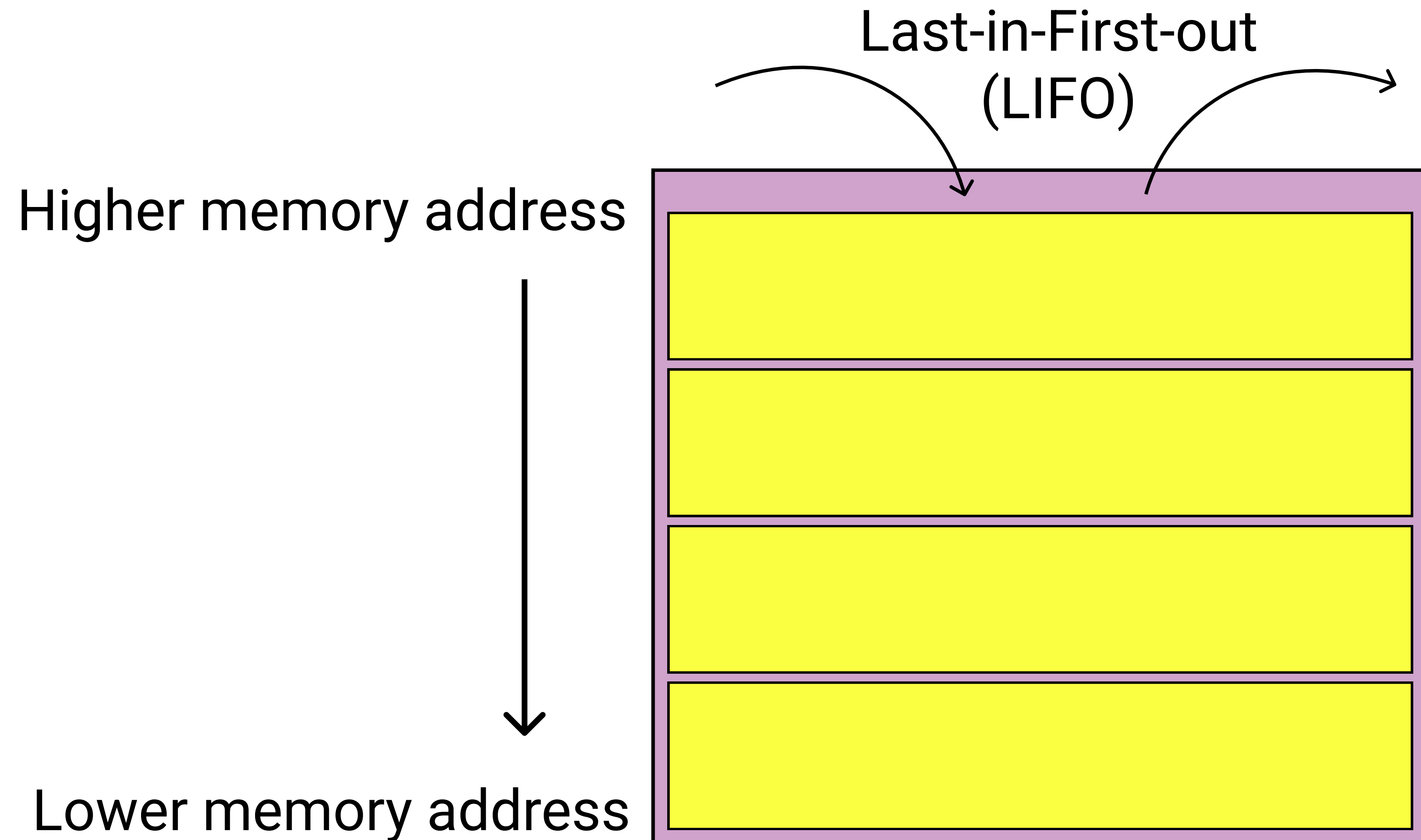
CPU Registers
(Fastest)

Registers

- General Purpose Registers: RAX, RBX, RCX, RDX
- Index Registers: RDI, RSI
- Pointers: RBP, RSP
- Program Counter: RIP



The Stack



x86-64 Assembly language: Part 1

Data and Arithmetic Instructions

mov	a, b	; a = b
lea	a, [b+c]	; a => (b+c)
add	a, b	; a = a + b
sub	a, b	; a = a - b

x86-64 Assembly language: Part 1

Data and Arithmetic Instructions

```
mov    rax, rbx
lea    rax, [rbp + 40]

add    rbx, rcx
sub    rax, rcx
```

Variants

```
mov    rax, 4

add    rax, 2
sub    rbx, 0x40
```

x86-64 Assembly language: Part 2

Program Flow Instructions

```
jmp    a    ; direct program flow to a
```

```
cmp    a, b    ; compare two values
```

```
jle    c    ; direct program flow to c
        ; if a ≤ b
```

x86-64 Assembly language: Part 2

Program Flow Instructions

jmp 0x506a4d92

cmp rax, rbx

jle 0x506a4e21

Variants

je 0x506a4e21

jbe 0x506a4e21

jne 0x506a4e21

Discussion and Resources

<https://www.compileenrun.com/docs/fundamental/os/memory-management/memory-hierarchy/>

<https://www.felixcloutier.com/x86/>

<https://x64.syscall.sh/>